Explainability Layer

gommage · June 2026

Abstract. gommage is not merely an application with an explainability feature. The whole project is the explainability layer for agent execution. It records what an agent saw, what it inferred, which tools it called, which evidence supported each step, and how the execution unfolded. The central artifact is the *Agent Execution Record* (AER), which stores reasoning provenance as a structured object and enables the execution to be visualized as a directed acyclic graph (DAG).

Purpose

The purpose of the explainability layer is to make autonomous agent behavior inspectable after execution. Traditional logs show events, but they rarely show the reasoning context that caused an event. gommage captures the agent's trajectory as structured provenance so developers, auditors, and operators can answer three questions:

1. What did the agent know at each step?
2. Why did the agent choose the next action?
3. Which tool calls, observations, and evidence led to the final outcome?

Agent Execution Record

The AER is the provenance object produced for an agent run. It groups the run metadata and the ordered sequence of execution steps into a normalized schema. At the run level, an AER contains identifiers and lifecycle state such as:

- `run_id` : the unique execution identifier.
- `jira_ticket_id` : the originating Jira ticket or task context.
- `agent_name` : the agent that produced the trace.
- `status` : the execution state, such as running, completed, or failed.
- `started_at` and `completed_at` : timestamps for the run.
- `steps` : the sequence of recorded reasoning and action steps.

Each AER step records the local reasoning state. The step schema supports LLM steps, tool steps, observations, and decisions. A step includes:

- `intent`: what the agent was trying to do.
- `observation`: what the agent observed at that point.
- `inference`: the reasoning or interpretation produced from the observation.
- `context`: the local state visible to the agent.
- `llm`: prompt, system message, model, response, latency, and token count.
- `tool`: tool name, parameters, result, latency, errors, and mock status.
- `evidence`: links to supporting artifacts, verdicts, and confidence.
- `timestamp`: the time at which the step was recorded.

Reasoning Provenance Features

Feature	Explainability Value
Prompt and response capture	Shows exactly what the model received and produced at each LLM step.
Context snapshots	Preserves the state available to the agent when it made a decision.
Intent, observation, inference fields	Separates the agent's goal, sensory input, and interpretation so reasoning can be audited step by step.
Tool call provenance	Records tool parameters, results, latency, errors, and whether a call was side-effecting or mocked.
Evidence chains	Connects steps to supporting artifacts, verdicts, confidence scores, and metadata.
Timing metadata	Supports latency analysis across LLM calls, tool calls, and idle time between steps.
Trace hashing	Provides a stable fingerprint for detecting whether a recorded trajectory has changed.

Execution DAG

The visualized DAG presents the AER as an execution graph. Nodes represent recorded reasoning, decision, LLM, tool, or observation steps. Edges represent the execution order and dependency between steps. This graph makes the agent's trajectory readable as a causal path rather than as a flat log.

The DAG view provides:

- A run-level view of the full execution trajectory.
- Step-level expansion into prompts, responses, tool calls, and evidence.
- Visual distinction between LLM reasoning, tool execution, observations, errors, and mocked side-effecting calls.
- Branch visibility when a replay diverges from the original execution.
- A direct path from graph nodes back to the underlying AER fields.

Why This Is the Explainability Layer

gommage provides explainability by sitting between the agent and the systems it uses. It captures model interaction, tool interaction, context, evidence, and execution timing without requiring the developer to reconstruct behavior from unstructured logs. The result is a provenance layer that supports debugging, audit, replay, and human inspection of agent reasoning.

The AER is the durable representation of that provenance. The DAG is the human interface for navigating it. Together, they make agent behavior explainable at the level where failures actually occur: individual reasoning and action steps.